

Conception et développement d'IHM

Cours S2: Vue d'ensemble de React

Introduction et prérequis: contenu

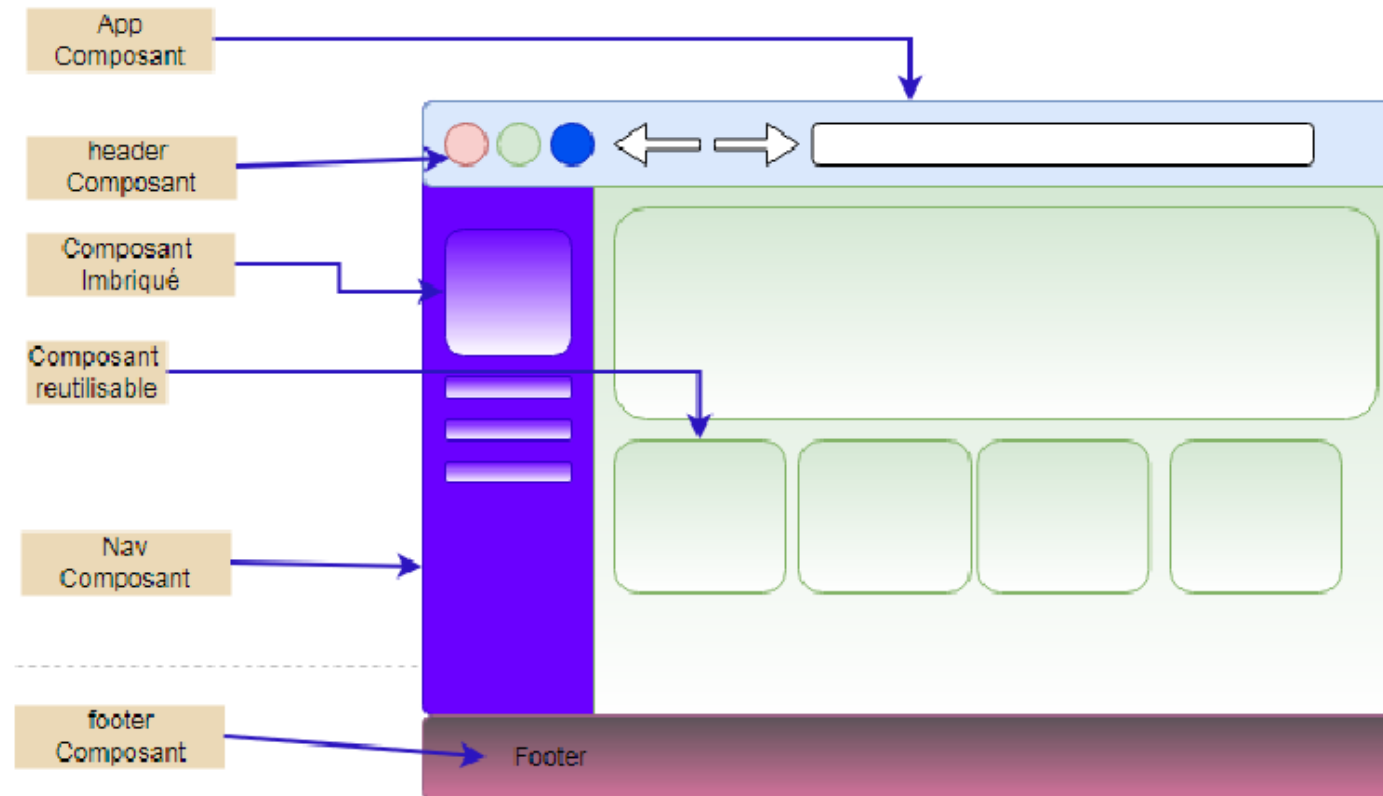
- Présentation de React
- L'écosystème React
- Architecture d'une application React
- MEP de l'environnement de développement et création de votre première application React
- La structure d'un projet React
- Le chargement initial d'une application React
- Une vue d'ensemble de React
- React VS JavaScript pur
- React Virtuel DOM
- Réfléchir en React

L'écosystème React / Architecture d'une application React

Présentation de React

React (Ou ReactJS, React.js) est une bibliothèque JavaScript développée par Facebook, utilisée pour créer des composants d'affichage réutilisables.

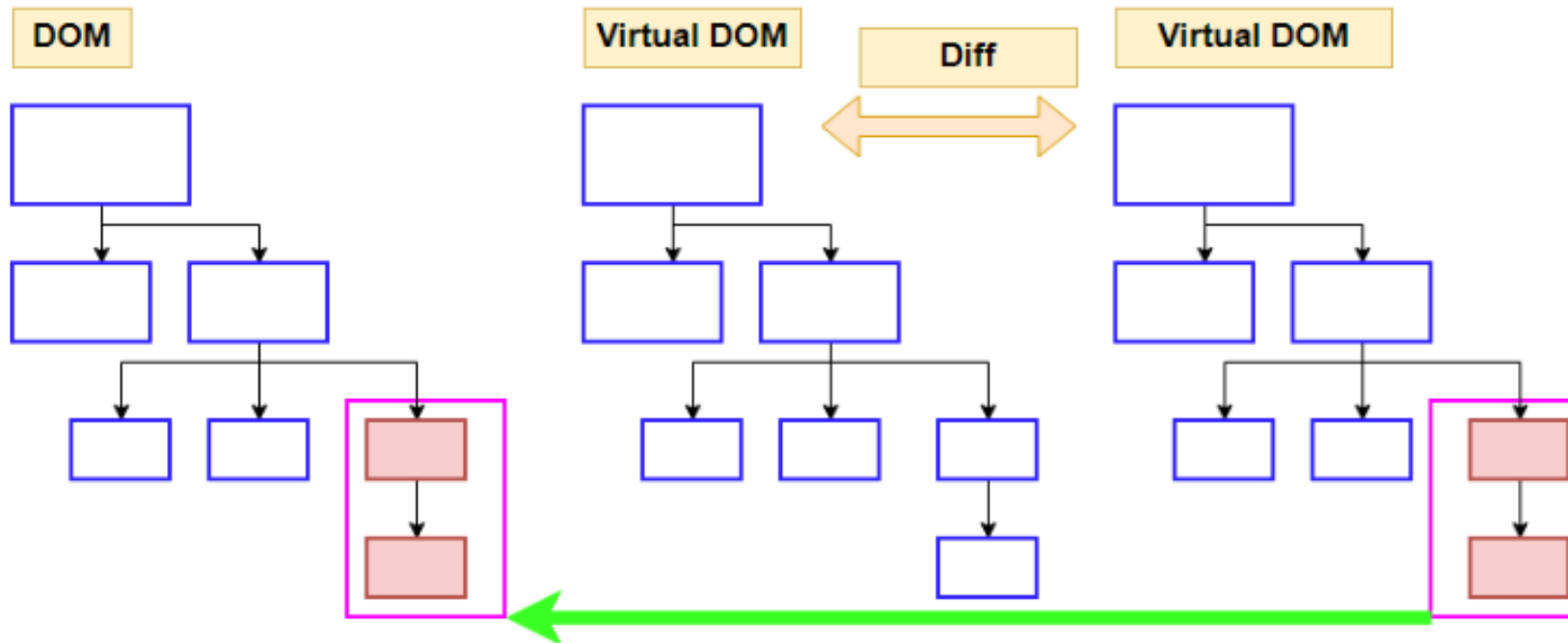
Pour créer une application avec React, on va donc créer des composants d'affichage (en général des classes ou des fonctions) qui seront ensuite assemblés afin de former l'application finale.



Présentation de React

L'intérêt de React, outre le fait de créer des composants réutilisables, et de permettre aussi une mise à jour rapide de la page HTML.

En effet, lors de l'utilisation de bibliothèques telles que jQuery (qui manipulent les éléments HTML de la page, c'est-à-dire le DOM, directement), les performances sont bien moindres car la manipulation du DOM directement en JavaScript est lente lors de l'exécution. React ne manipule pas le DOM directement mais une copie interne de celui-ci (les objets ou éléments React, appelés le DOM virtuel), et produit les modifications sur l'affichage uniquement lorsque cela s'avère nécessaire.



L'écosystème React: npm

npm, acronyme de **N**ode **P**ackage **M**anager, est le gestionnaire de paquets par défaut pour l'écosystème Node.js. Il permet aux développeurs JavaScript d'installer, de partager et de gérer les dépendances de leurs projets Node.js de manière efficace. C'est l'équivalent de composer pour le PHP.

Ci-dessous les fonctionnalités les plus importantes d'npm :

- **Installation de paquets** : npm permet d'installer des paquets JavaScript (bibliothèques, frameworks, outils, etc.) à l'aide de la commande `npm install`. Ces paquets sont généralement hébergés sur le registre npm public, mais peuvent également être installés à partir de sources privées ou locales.
- **Gestion des dépendances** : Les développeurs peuvent spécifier les dépendances de leurs projets dans un fichier `package.json`, qui décrit les métadonnées du projet ainsi que les dépendances requises. npm gère alors ces dépendances en installant automatiquement les paquets nécessaires et en les mettant à jour si nécessaire.
- **Scripts personnalisés** : npm permet également de définir des scripts personnalisés dans le fichier `package.json`, ce qui permet d'exécuter des tâches telles que la compilation de code, les tests, le déploiement, etc., en utilisant des commandes simples telles que `npm run script-name`.
- **Communauté et écosystème** : npm héberge l'un des plus vastes écosystèmes de paquets open source au monde, offrant des milliers de modules prêts à l'emploi pour une variété de tâches de développement. Cela favorise la réutilisabilité du code et accélère le processus de développement.

npm est un outil essentiel pour les développeurs JavaScript travaillant avec React

L'écosystème React: Babel

Babel est un compilateur JavaScript populaire à code source ouvert qui permet aux développeurs d'écrire du code JavaScript moderne en utilisant les dernières fonctionnalités du langage, puis de convertir ce code dans un format qui peut être exécuté sur des navigateurs ou des environnements plus anciens. Il est couramment utilisé dans le développement web pour s'assurer que le code écrit dans la dernière version de JavaScript est compatible avec un plus grand nombre de navigateurs.

Babel fonctionne en prenant le code JavaScript écrit à l'aide de nouvelles syntaxes et API de langage qui ne sont pas encore prises en charge par tous les navigateurs, et en le transformant en une version de JavaScript qui est largement prise en charge. Ci-dessous un exemple de transformation faite par Babel.

```
// ES6 code
const greet = (name) => {
  console.log(`Hello, ${name}!`);
}

greet('ALAMI');

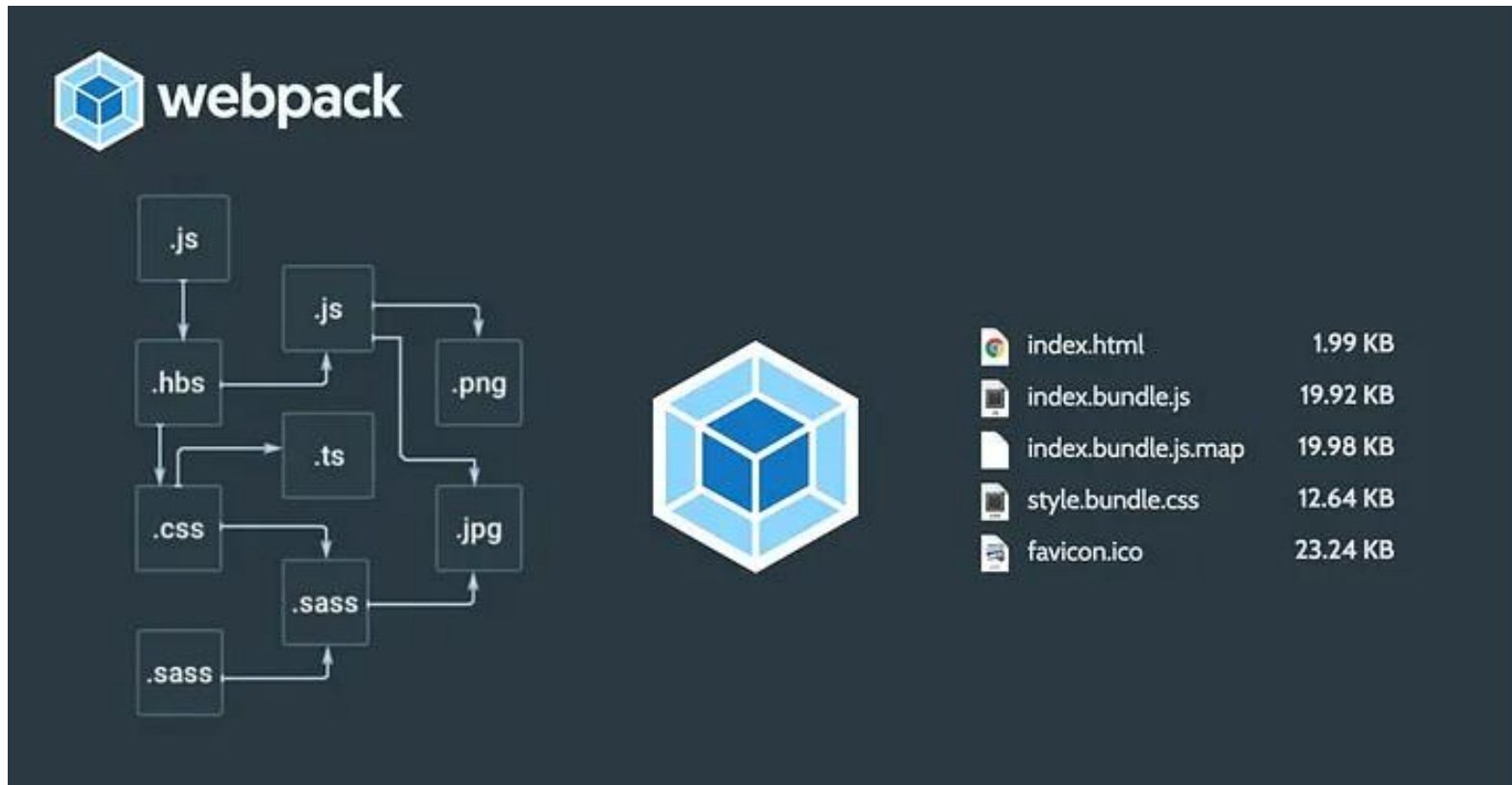
// Transpiled ES5 code
'use strict';

var greet = function greet(name) {
  console.log('Hello, ' + name + '!');
};

greet('ALAMI');
```

L'écosystème React: Bundlers

Le **bundler** condense tout le code de votre application pour n'avoir qu'un seul fichier JavaScript à importer dans votre page HTML. L'intérêt majeur c'est que vous n'êtes plus contraints d'avoir un fichier volumineux quand vous codez avec JavaScript, où il est impossible de retrouver votre fonction. Vous pouvez faire des modules (fichiers isolés), y faire référence dans un autre fichier et tout de même être capable de les utiliser dans le navigateur sans ajouter une balise `<script>` à chaque fois que vous ajoutez un fichier. Les deux bundlers majeurs sur le marché actuellement sont **Webpack** et **Vite**

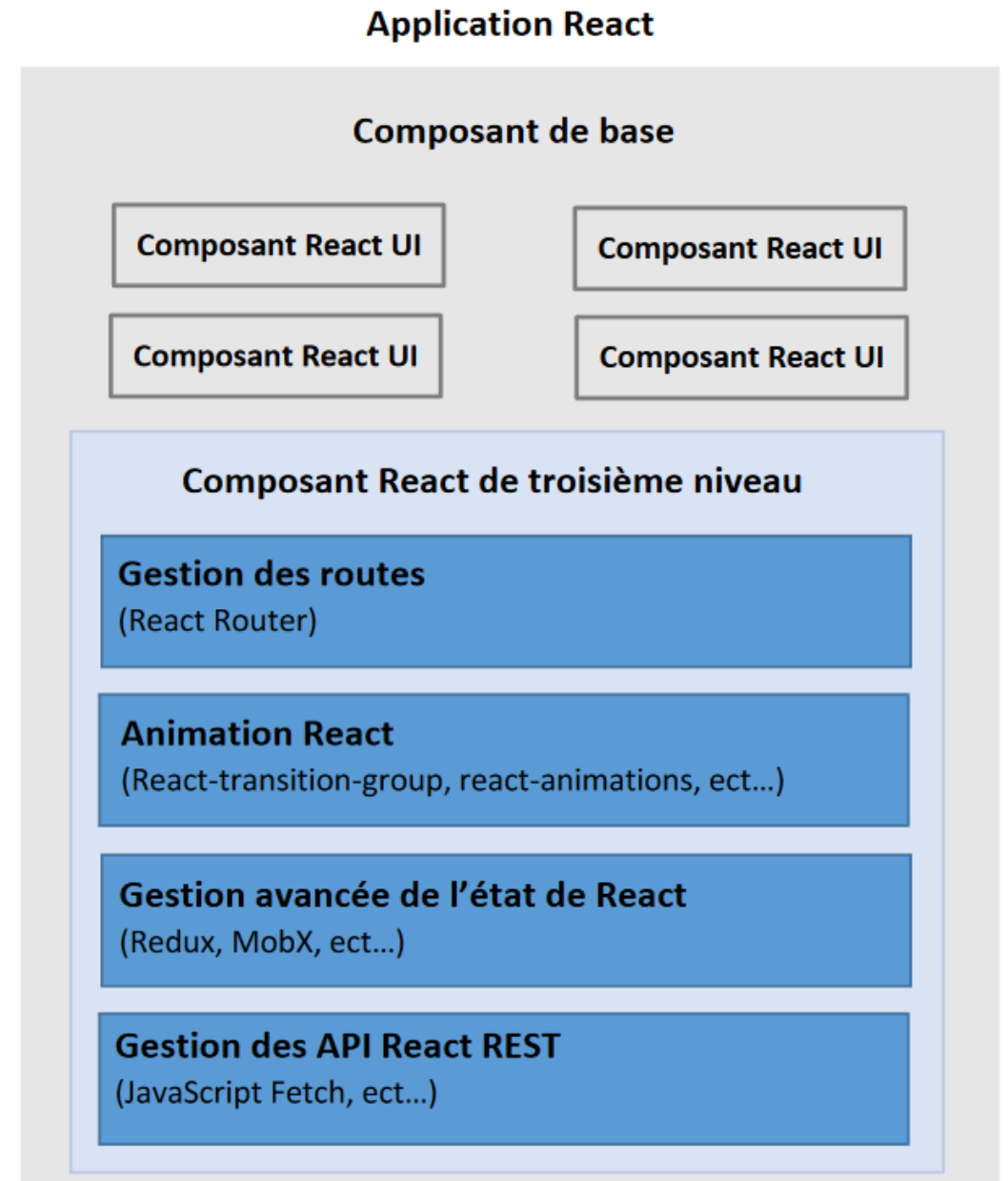


Architecture d'une application React

L'architecture de React repose sur des concepts comme : le composant d'ordre supérieur, le contexte, les accessoires de rendu, les références, React Hooks, etc.

Examinons l'architecture d'une application React:

- L'application React démarre avec un seul composant racine.
- Le composant racine est créé grâce à un ou plusieurs composants.
- Chaque composant peut être composé d'autres composants.
- L'application React a la possibilité d'ajouter un composant tiers à des fins spécifiques comme le routage, l'animation, la gestion de l'état, etc

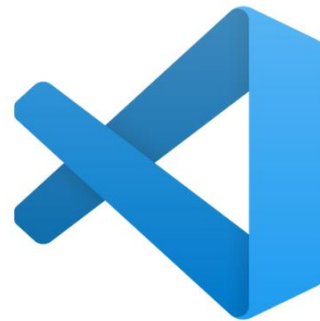


MEP de l'environnement de développement et création de votre première application React

L'environnement de développement React

Pour développer avec React, vous avez besoin des logiciels ci-dessous:

- **Node.js** : Les applications React sont généralement développées en utilisant Node.js, un environnement d'exécution JavaScript. Vous devrez installer Node.js pour exécuter JavaScript sur votre machine et utiliser npm (Node Package Manager) pour gérer les packages et les dépendances.
- **npm ou Yarn** : npm est inclus avec Node.js et est le gestionnaire de packages par défaut pour JavaScript. Vous pouvez également utiliser Yarn, un autre gestionnaire de packages populaire pour Node.js. Vous utiliserez npm ou Yarn pour installer React et d'autres packages nécessaires à votre projet.
- **Éditeur de code** : Les choix populaires incluent Visual Studio Code, Sublime Text, Atom ou WebStorm.
- **L'extension navigateur « React Developer Tools » (optionnel)** : Il s'agit d'une extension de navigateur disponible pour Chrome, Firefox, et Edge qui vous permet d'inspecter les hiérarchies de composants React et de déboguer les applications React.



**React
Developer
Tools**

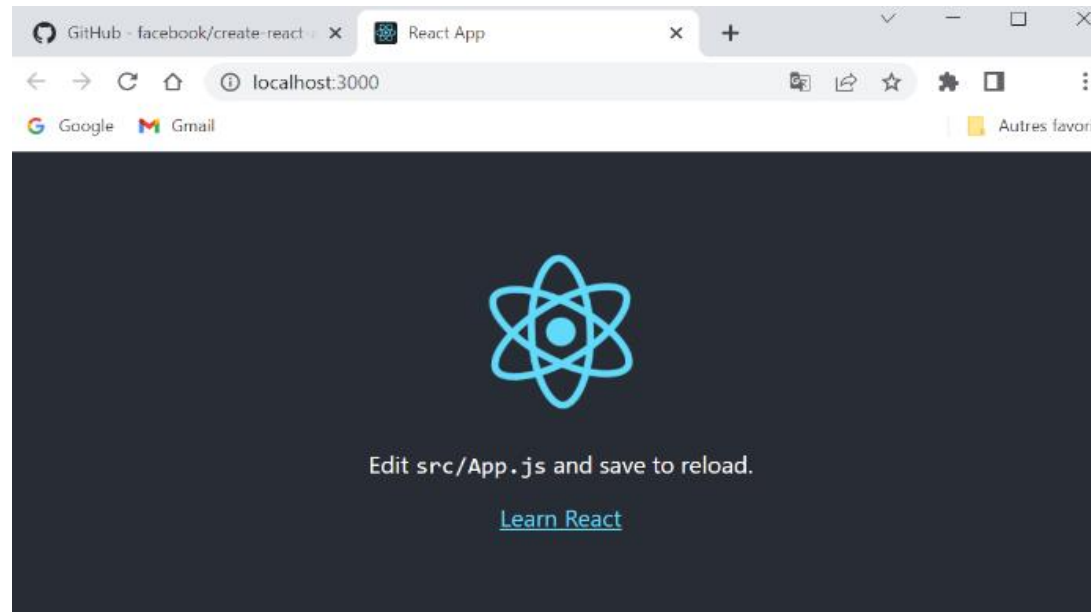
Création de votre première application React

La façon la plus simple et la plus pratique pour créer une application React est d'utiliser l'utilitaire en ligne de commande **create-react-app** (<https://github.com/facebook/create-react-app>). Cette utilitaire est créé par l'équipe React et permet de créer la squelette d'une application React en une seule commande.

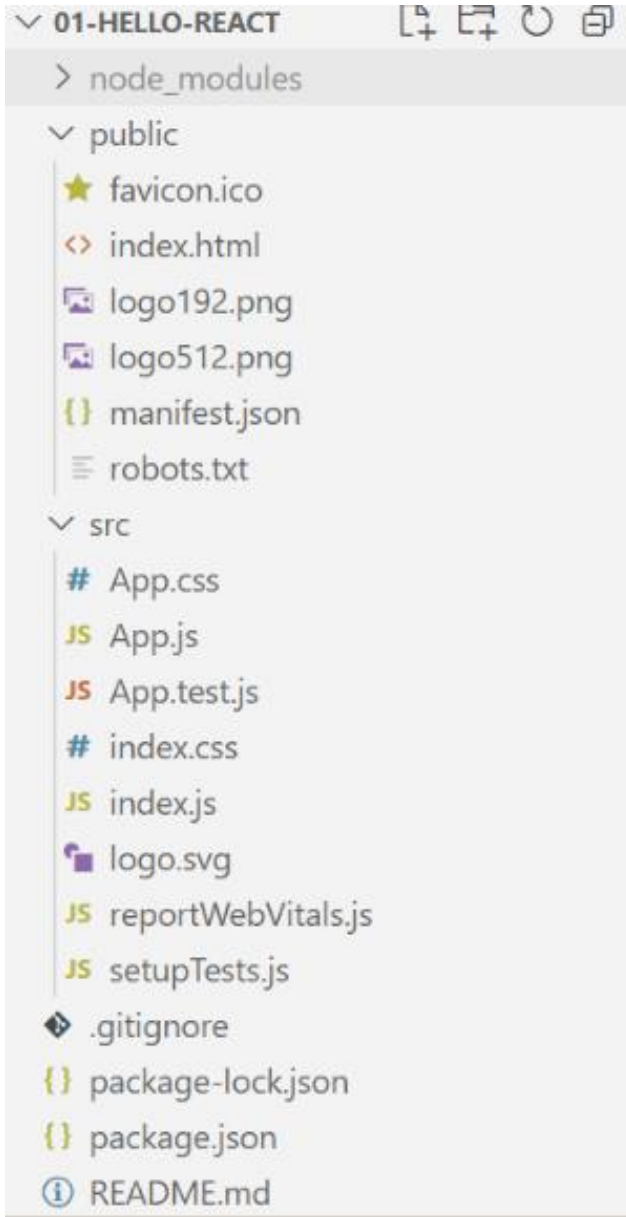
Pour créer votre premier projet, exécuter les commandes suivantes :

```
npx create-react-app nom-votre-app-react  
cd nom-votre-app-react  
npm start
```

Ouvrir <http://localhost:3000> pour accéder à votre projet



La structure d'un projet React



- Le dossier **node_modules** contient toutes dépendances JavaScript de React
- Le dossier **public** vous y trouverez le fichier **index.html** qui est le point de départ de l'application avec la **div** qui a pour class **root** qui permet de dire a React ou est ce qu'il doit venir s'ancrer.
- Le fichier **.gitignore** a pour rôle de signaler à **git** les fichiers qui ne doivent pas être envoyé au repo distant notamment le dossier node_module.
- Le fichier **package.json** contient plusieurs informations sur votre application React notamment les scripts, les dépendances.
- Le fichier **Readme** permet de documenter votre app en Markdown.
- Le dossier **src** est l'endroit où toute la magie de votre application réside, on y retrouve le fichier **index.js** qui est le fichier qui charge notre premier composant, le composant **App** qui est ensuite greffer a la **div** avec la class **root** qui se trouve dans le fichier **index.html** au niveau du dossier public

Le chargement initial d'une application React

Quand vous créez un projet React avec la commande **create-react-app**, il y a deux fichiers principaux qui vont être créés, le fichier **public/index.html** et le fichier **src/index.js**

Commençons par le fichier public/index.html:

C'est là qu'est défini la structure de base de la page HTML principale de l'application, plus important il y est défini une **div** avec l'id **root**. C'est en effet dans cette div que notre application sera afficher.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
  </body>
</html>
```

Le chargement initial d'une application React

Passons ensuite au fichier src/index.js:

Ce fichier va permettre de récupérer nos composants et les afficher dans la **div#root** de **public/index.html**

```
1. import React from 'react';
2. import ReactDOM from 'react-dom/client';
3. import './index.css';
4. import App from './App';
5.
6. const root = ReactDOM.createRoot(document.getElementById('root'));
7. root.render(
8.   <React.StrictMode>
9.     <App />
10.   </React.StrictMode>
11.);
```

- **La ligne 1** : importe la bibliothèque React. Comme React transforme l'instruction JSX que nous écrivons en `React.createElement()`, tous les composants React doivent importer le module React. Si vous ignorez cette étape, votre application produira une erreur.
- **La ligne 2** : importe Le module `react-dom` qui fournit des méthodes spécifiques au DOM que vous pouvez utiliser au niveau racine de votre application.
- **La ligne 3** : importe la feuille de style `src/index.css`
- **La ligne 4** : importe le fichier `src/App.js` qui définit une fonction nommée `App`
- **La ligne 6** : affiche l'élément React `<App />` au sein du nœud DOM `root` et renvoie une référence sur le composant.

Le chargement initial d'une application React

Passons maintenant au composant App (src/App.js) :

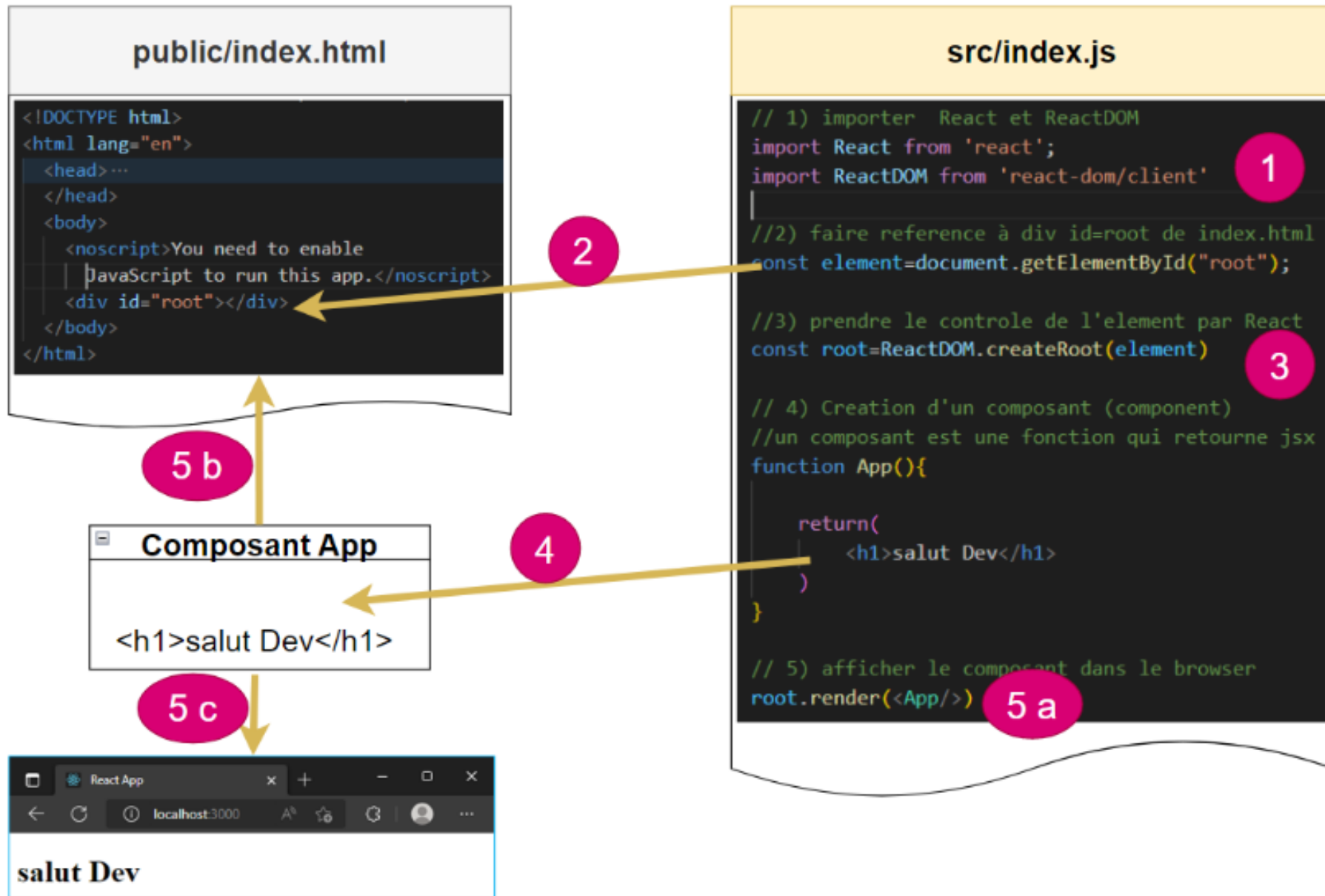
```
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>Edit <code>src/App.js</code> and save to reload.</p>
        <a className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer">
          Learn React
        </a>
      </header>
    </div>
  );
}

export default App;
```

La fonction **App** renvoie une expression JSX. Cette expression définit ce que votre navigateur restitue finalement au DOM. Tout en bas du fichier App.js, l'instruction `export default App` rend notre composant App disponible pour les autres modules

Le chargement initial d'une application React: récap



Création et imbrication de composants

Passons maintenant au composant App (src/App.js) :

Les applications React sont constituées de composants. Un composant est un élément de l'interface utilisateur qui possède sa propre logique et sa propre apparence. Un composant peut être aussi petit qu'un bouton ou aussi grand qu'une page entière. Les composants React sont des fonctions JavaScript qui renvoient des balises :

```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}
```

Maintenant qu'on a déclaré le composant **MyButton**, on peut l'imbriquer dans un autre composant.

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Welcome to my app</h1>  
      <MyButton />  
    </div>  
  );  
}
```

Les noms des composants React doivent toujours commencer par une lettre majuscule, tandis que les balises HTML doivent être en minuscules.

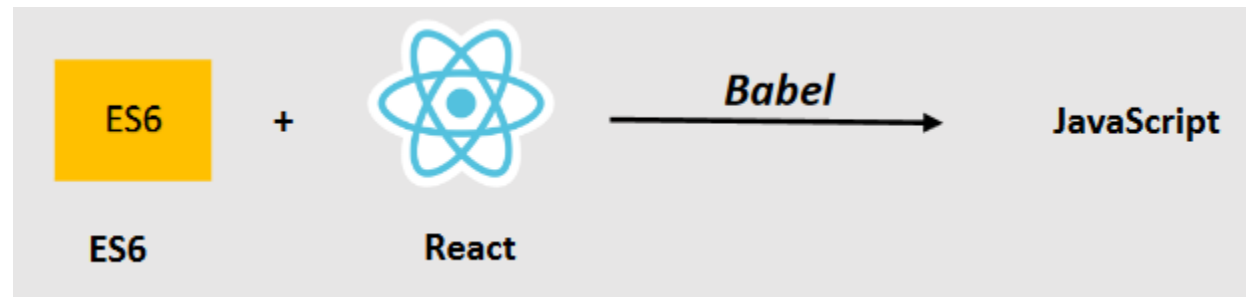
Écrire des balises avec JSX

La syntaxe de balisage utilisée dans React est appelée **JSX** (avaScript eXtension). Il s'agit d'une syntaxe plus strict que HTML:

1. Vous devez fermer les balises comme `<br />`
2. Votre composant ne peut pas renvoyer plusieurs balises JSX. Vous devez les intégrer dans un parent partagé, comme un `<div>...</div>` ou un wrapper `<>...</>` vide.

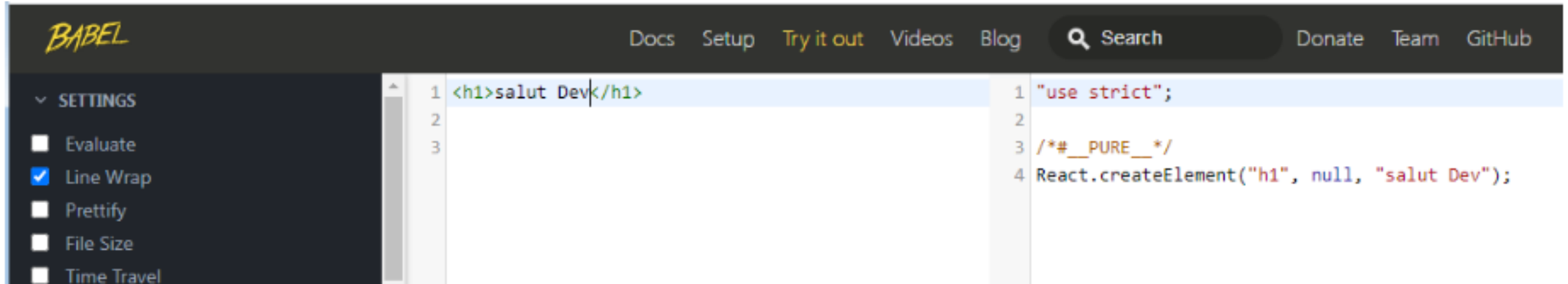
```
function AboutPage() {  
  return (  
    <>  
      <h1>About</h1>  
      <p>Hello there.<br />How do you do?</p>  
    </>  
  );  
}
```

L'outil **Babel** permet d'interpréter le code JSX et de le transformer en interne en code JavaScript compréhensible par le navigateur.

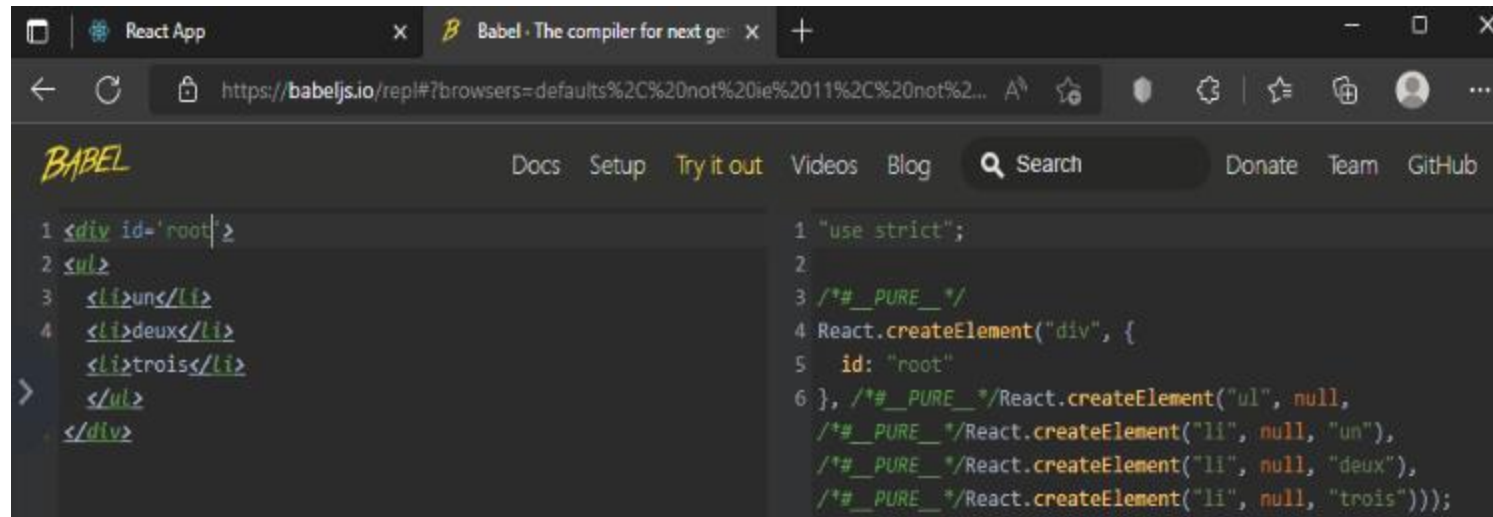


Écrire des balises avec JSX: comment se fait l'interprétation de JSX

L'outil online: babeljs.io/repl permet d'illustrer en quoi votre JSX est transformé



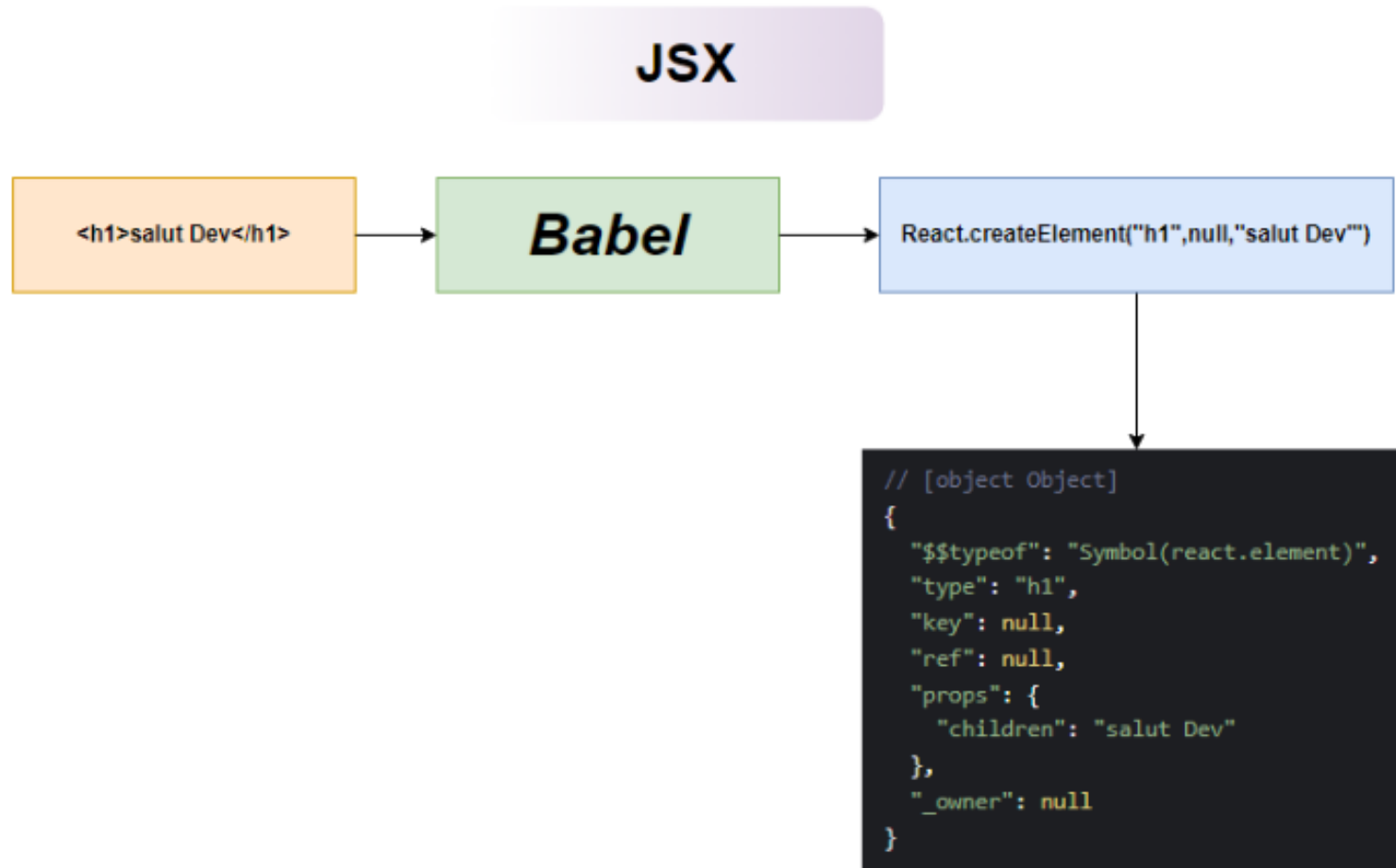
The screenshot shows the Babel REPL interface. On the left, there's a sidebar with the BABEL logo and a SETTINGS menu with options: Evaluate (unchecked), Line Wrap (checked), Prettify (unchecked), File Size (unchecked), and Time Travel (unchecked). The main area is split into two panes. The left pane shows the input JSX: `1 <h1>salut Dev</h1>`. The right pane shows the transformed JavaScript code: `1 "use strict";`, `2`, `3 /*#__PURE__*/`, and `4 React.createElement("h1", null, "salut Dev");`.



The screenshot shows the Babel REPL interface with a browser window open at <https://babeljs.io/repl#?browsers=defaults%2C%20not%20ie%2011%2C%20not%20...>. The left pane shows a nested JSX structure: `1 <div id='root'>`, `2 <ul>`, `3 <li>un</li>`, `4 <li>deux</li>`, `<li>trois</li>`, `</ul>`, and `</div>`. The right pane shows the transformed JavaScript code, which is more complex due to the nesting: `1 "use strict";`, `2`, `3 /*#__PURE__*/`, `4 React.createElement("div", {`, `5 id: "root"`, `6 }, /*#__PURE__*/React.createElement("ul", null,`, `/*#__PURE__*/React.createElement("li", null, "un"),`, `/*#__PURE__*/React.createElement("li", null, "deux"),`, `/*#__PURE__*/React.createElement("li", null, "trois")));`

Le code JSX est compréhensible alors que le code JavaScript correspondant est difficile à comprendre à cause d'imbrications d'appel de `createElement`. JSX facilite le travail de développeur.

Écrire des balises avec JSX: comment se fait l'interprétation de JSX



Le code JSX `<h1> salut Dev </h1>` est transformé par la Bibliothèque Babel en code javascript `React.createElement("h1",null,"salut Dev")` qui retourne un objet javascript. Cet objet contient toutes les informations nécessaires à React pour afficher l'élément dans le navigateur. Par exemple `'type' : 'h1'` précise le type de l'élément et `'props' : {'children' : 'salut Dev'}` précise le texte contenu dans h1

Ajout de styles

Dans React, vous spécifiez une classe CSS avec **className**. Cela fonctionne de la même manière que l'attribut HTML class :

```
<img className="avatar" />
```

Vous écrivez ensuite les règles CSS correspondantes dans un fichier CSS distinct :

```
.avatar {  
  border-radius: 50%;  
}
```

React ne prescrit pas la manière dont vous ajoutez les fichiers CSS. Dans le cas le plus simple, vous ajouterez une balise <link> à votre HTML

Affichage des données

JSX vous permet d'insérer des balises dans JavaScript. Les accolades vous permettent de "revenir en arrière" dans JavaScript afin d'intégrer une variable de votre code et de l'afficher à l'utilisateur. Par exemple, ceci affichera user.name :

```
return (  
  <h1>  
    {user.name}  
  </h1>  
);
```

Il est également possible de "s'échapper en JavaScript" à partir d'attributs JSX, mais il faut utiliser des accolades au lieu de guillemets. Par exemple, className="avatar" transmet la chaîne "avatar" en tant que classe CSS, mais src={user.imageUrl} lit la valeur de la variable JavaScript user.imageUrl, puis transmet cette valeur en tant qu'attribut src :

```
return (  
  <img  
    className="avatar"  
    src={user.imageUrl}  
  />  
);
```

Rendu conditionnel

Dans React, il n'y a pas de syntaxe spéciale pour écrire des conditions. Au lieu de cela, vous utiliserez les mêmes techniques que lorsque vous écrivez du code JavaScript normal. Par exemple, vous pouvez utiliser une instruction if pour inclure JSX de manière conditionnelle :

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return (
  <div>
    {content}
  </div>
);
```

Ou avec l'opérateur ternaire:

```
<div>
  { isLoggedIn ? <AdminPanel /> : <LoginForm /> }
</div>
```

Affichage des listes

Vous vous appuyerez sur des fonctionnalités JavaScript telles que la boucle for et la fonction `Array.map()` pour afficher les listes. Par exemple, supposons que vous avez un tableau de produits :

```
const products = [ { title: 'Cabbage', id: 1 }, { title: 'Garlic', id: 2 }, { title: 'Apple', id: 3 }, ];
```

Dans votre composant, utilisez la fonction `map()` pour transformer un tableau de produits en un tableau d'éléments `<li>` :

```
const listItems = products.map(product =>
  <li key={product.id}>
    {product.title}
  </li>
);

return (
  <ul>{listItems}</ul>
);
```

Remarquez que `<li>` possède un attribut `key`. Pour chaque élément d'une liste, vous devez fournir une chaîne de caractères ou un nombre qui identifie de manière unique cet élément parmi ses frères et sœurs. En général, une clé doit provenir de vos données, comme l'identifiant d'une base de données. React utilise vos clés pour savoir ce qui s'est passé si vous insérez, supprimez ou réorganisez les éléments par la suite.

Réagir aux événements

Vous pouvez répondre aux événements en déclarant des fonctions de gestion d'événements à l'intérieur de vos composants :

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

Remarquez que `onClick={handleClick}` n'a pas de parenthèses à la fin ! N'appellez pas la fonction de gestion de l'événement : vous devez seulement la transmettre. React appellera votre gestionnaire d'événement lorsque l'utilisateur cliquera sur le bouton.

Mise à jour de l'écran

Souvent, vous voudrez que votre composant se "souviene" de certaines informations et les affiche. Par exemple, vous voudrez peut-être compter le nombre de fois qu'un bouton est cliqué. Pour ce faire, ajoutez un état (State) à votre composant.

Tout d'abord, importez **useState** de React :

```
import { useState } from 'react';
```

Vous pouvez maintenant déclarer une variable d'état dans votre composant :

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  // ...  
}
```

Vous obtiendrez deux choses de `useState` : l'état actuel (`count`) et la fonction qui vous permet de le mettre à jour (`setCount`). Vous pouvez leur donner n'importe quel nom, mais la convention est d'écrire [`something`, `setSomething`].

Mise à jour de l'écran

La première fois que le bouton est affiché, le compte sera de 0 car vous avez passé 0 à `useState()`. Lorsque vous souhaitez changer d'état, appelez `setCount()` et passez-lui la nouvelle valeur. Cliquer sur ce bouton incrémentera le compteur :

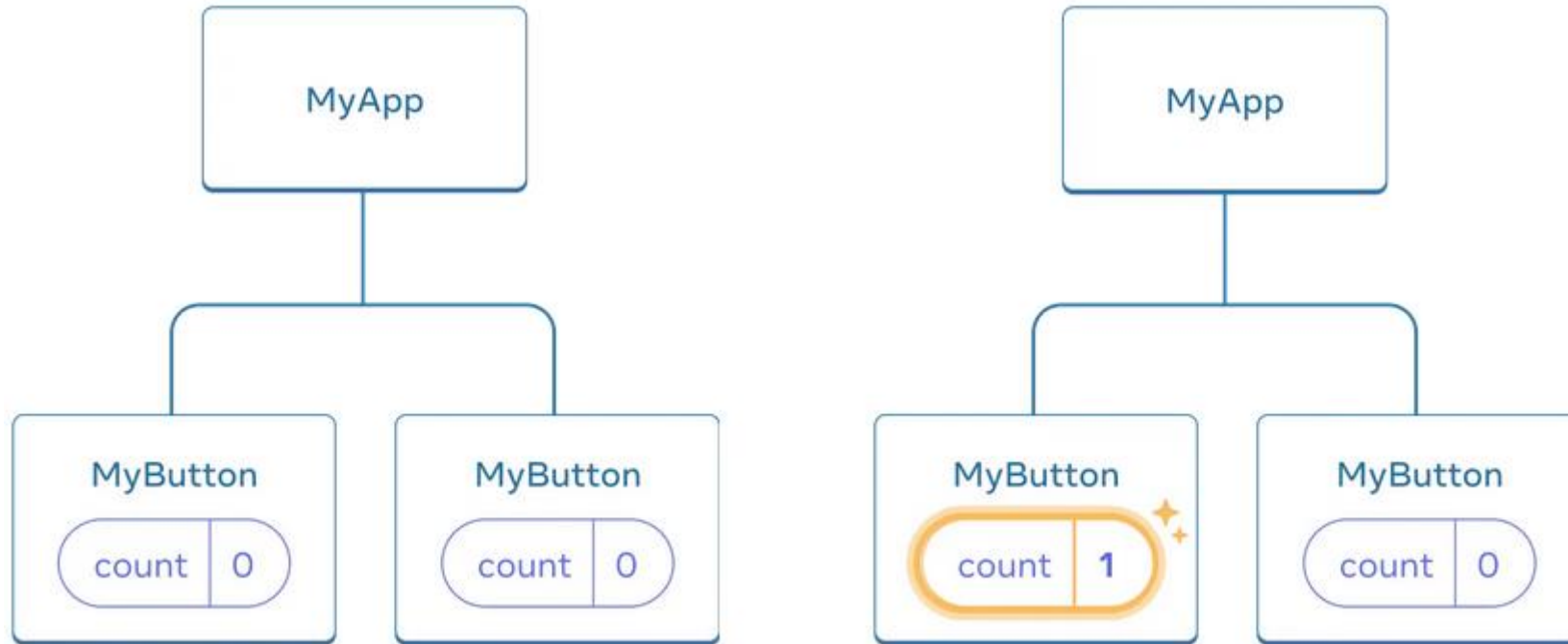
```
function MyButton() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Clicked {count} times  
    </button>  
  );  
}
```

React appellera à nouveau la fonction de votre composant. Cette fois, le compte sera de 1. Puis ce sera 2. Et ainsi de suite.

Si vous affichez le même composant plusieurs fois, chacun aura son propre état.

Partage des données entre les composants

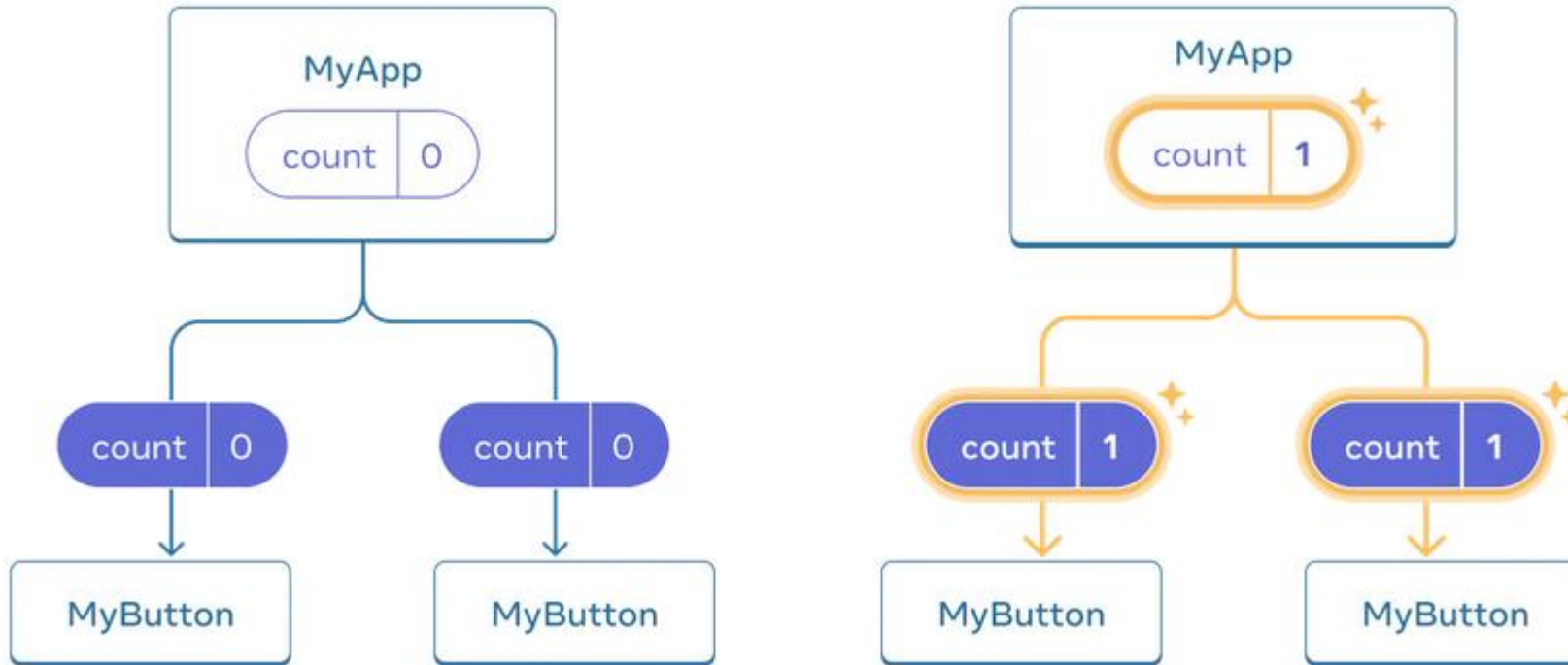
Dans le slide précédent, chaque composant MyButton avait son propre état (state) indépendant, et lorsque l'on cliquait sur chaque bouton, seul l'état du bouton cliqué changeait :



Cependant, vous aurez souvent besoin de composants qui partagent des données et qui sont toujours mis à jour ensemble.

Partage des données entre les composants

Pour que les deux composants MyButton affichent le même nombre et se mettent à jour ensemble, vous devez déplacer l'état (state) des boutons individuels "vers le haut", vers le composant le plus proche qui les contient tout les deux.



Désormais, lorsque vous cliquez sur l'un ou l'autre des boutons, l'état de MyApp change, ce qui modifie les deux états de MyButton.

Partage des données entre les composants

Voici comment vous pouvez exprimer cela en code. Tout d'abord, déplacez l'état de MyButton vers MyApp :

```
export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Counters that update separately</h1>
      <MyButton />
      <MyButton />
    </div>
  );
}

function MyButton() {
  // ... nous déplaçons le code d'ici et nous le mettons dans MyApp ...
}
```

Partage des données entre les composants

Ensuite, transmettez l'état de MyApp à chaque MyButton, ainsi que le gestionnaire de clic partagé. Vous pouvez transmettre des informations à MyButton en utilisant les accolades JSX.

```
export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Compteurs qui se mettent à jour ensemble</h1>
      <MyButton count={count} onClick={handleClick} />
      <MyButton count={count} onClick={handleClick} />
    </div>
  );
}
```

Les informations que vous transmettez de cette manière sont appelées "**props**". Le composant MyApp contient l'état de comptage et le gestionnaire d'événement handleClick, et les transmet tous deux à chacun des boutons en tant que **props**.

Partage des données entre les composants

Enfin, modifiez MyButton pour qu'il lise les propriétés que vous lui avez transmis depuis son composant parent :

```
function MyButton({ count, onClick }) {  
  return (  
    <button onClick={onClick}>  
      Clicked {count} times  
    </button>  
  );  
}
```

Lorsque vous cliquez sur le bouton, le gestionnaire onClick se déclenche. La propriété onClick de chaque bouton a été définie sur la fonction handleClick de MyApp, de sorte que le code qu'elle contient s'exécute. Ce code appelle setCount(count + 1), qui incrémente la variable d'état count. La nouvelle valeur de comptage est transmise à chaque bouton en tant que propriété, de sorte qu'ils affichent tous la nouvelle valeur. C'est ce qu'on appelle "élever l'état" (lifting state up). En déplaçant l'état vers le haut, vous l'avez partagé entre les composants.

React vs JavaScript pur

Ci-dessous à gauche, le code final avec React, à droite, le code équivalent en JavaScript pur.

```
import { useState } from 'react';

export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Compteurs qui se mettent à jour ensemble</h1>
      <MyButton count={count} onClick={handleClick} />
      <MyButton count={count} onClick={handleClick} />
    </div>
  );
}

function MyButton({ count, onClick }) {
  return (
    <button onClick={onClick}>
      Clicked {count} times
    </button>
  );
}
```

```
let count = 0;

function handleClick() {
  count++;
  render(); // Appel de la fonction qui met à jour l'interface
}

function createButton() {
  const button = document.createElement("button");
  button.textContent = `Clicked ${count} times`;
  button.addEventListener("click", handleClick);
  return button;
}

function createApp() {
  const app = document.createElement("div");

  const h1 = document.createElement("h1");
  h1.textContent = "Counters that update together";
  app.appendChild(h1);

  const button1 = createButton();
  const button2 = createButton();

  app.appendChild(button1);
  app.appendChild(button2);

  return app;
}

// Fonction de rendu de l'application
function render() {
  const root = document.getElementById("root");
  root.innerHTML = "";
  root.appendChild(createApp());
}

// Rendu initial
render();
```

React vs JavaScript pur

- Avec JavaScript pur, nous avons besoin de 13 lignes de code de plus pour faire la même chose qu'avec React.
- Avec JSX, on évite la syntaxe JS verbeuse de création d'éléments HTML (`document.createElement()`, `appendChild()`,...)
- Avec React, chaque composant garde par défaut son propre état (state), chose qui est difficile à faire avec JavaScript pur.
- L'avantage majeur de React par rapport à JavaScript pur est qu'on est plus obligé de mettre à jour le DOM manuellement après chaque changement. React prend en charge de mettre à jour l'interface automatiquement et d'une manière optimale (avec le minimum de modifications, afin d'améliorer les performances de l'application).

```
let count = 0;

function handleClick() {
  count++;
  render(); // Appel de la fonction qui met à jour l'interface
}

function createButton() {
  const button = document.createElement("button");
  button.textContent = `Clicked ${count} times`;
  button.addEventListener("click", handleClick);
  return button;
}

function createApp() {
  const app = document.createElement("div");

  const h1 = document.createElement("h1");
  h1.textContent = "Counters that update together";
  app.appendChild(h1);

  const button1 = createButton();
  const button2 = createButton();

  app.appendChild(button1);
  app.appendChild(button2);

  return app;
}

// Fonction de rendu de l'application
function render() {
  const root = document.getElementById("root");
  root.innerHTML = "";
  root.appendChild(createApp());
}

// Rendu initial
render();
```

React Virtuel DOM

React se distingue par son modèle d'objet de document (DOM) virtuel, qui présente ses excellentes fonctionnalités:

- **Performance améliorée** : Au lieu de mettre à jour le DOM réel à chaque modification de l'état ou des données, React effectue d'abord les mises à jour sur une représentation virtuelle du DOM. Ensuite, il compare cette représentation virtuelle avec l'ancienne pour déterminer les changements réels à apporter au DOM, puis il ne met à jour que les parties affectées. Cela réduit le coût des opérations de manipulation du DOM, ce qui se traduit par des performances améliorées de l'application.
- **Optimisation des rendus** : Au lieu de mettre à jour le DOM après chaque modification de l'état ou des données, React peut différer les mises à jour et les regrouper pour minimiser les opérations coûteuses.
- **Compatibilité multiplateforme** : Le Virtual DOM permet à React de fonctionner de manière cohérente sur différentes plateformes, y compris le navigateur Web, les applications mobiles et même les environnements côté serveur. Cela offre une expérience de développement uniforme et facilite le partage de code entre les différentes plateformes.
- **Facilité de développement** : Le Virtual DOM simplifie le processus de développement en permettant aux développeurs de se concentrer sur la logique métier de leur application plutôt que sur les manipulations directes du DOM.
- **Facilité de tests** : Le Virtual DOM facilite les tests automatisés en permettant aux développeurs de simuler le rendu de composants React dans un environnement de test isolé.

Réfléchir en React

React peut changer votre façon de concevoir les designs que vous regardez et les applications que vous créez. Lorsque vous construisez une interface utilisateur avec React, vous la décomposez d'abord en éléments appelés composants. Ensuite, vous décrivez les différents états visuels de chacun de vos composants. Enfin, vous connecterez vos composants entre eux de manière à ce que les données les traversent.

Pour mettre en œuvre une interface utilisateur dans React, vous suivrez généralement les cinq étapes suivantes:

Étape 1 : Décomposer l'interface utilisateur en une hiérarchie de composants

Commencez par dessiner des boîtes autour de chaque composant et sous-composant de la maquette et nommez-les. Vous pouvez envisager de diviser une conception en composants de différentes manières :

Programmation : utilisez les mêmes techniques pour décider si vous devez créer une nouvelle fonction ou un nouvel objet. L'une de ces techniques est le principe de la responsabilité unique, c'est-à-dire qu'un composant ne devrait idéalement faire qu'une seule chose. S'il finit par s'étendre, il doit être décomposé en sous-composants plus petits.

CSS : considérez ce pour quoi vous créeriez des sélecteurs de classe. (Toutefois, les composants sont un peu moins granulaires).

Conception : réfléchissez à la manière dont vous organiseriez les couches de la conception.

Réfléchir en React

Étape 2 : Construire une version statique dans React

Maintenant que vous avez votre hiérarchie de composants, il est temps d'implémenter votre application. L'approche la plus simple consiste à construire une version qui rend l'interface utilisateur à partir de votre modèle de données sans ajouter d'interactivité. Il est souvent plus facile de construire d'abord la version statique et d'ajouter l'interactivité plus tard.

Pour construire une version statique de votre application, vous devrez construire des composants qui réutilisent d'autres composants et transmettent des données à l'aide de « props ».

Vous pouvez construire « top down » en commençant par construire les composants situés plus haut dans la hiérarchie ou « bottom up » en travaillant à partir des composants situés plus bas. Dans les applications les plus simples, il est généralement plus facile d'aller de haut en bas, et dans les projets plus importants, il est plus facile d'aller de bas en haut.

Étape 3 : Trouver la représentation minimale mais complète de l'état de l'interface utilisateur

Pour rendre l'interface utilisateur interactive, vous devez permettre aux utilisateurs de modifier votre modèle de données sous-jacent. Pour ce faire, vous utiliserez les états « state ». Le principe le plus important pour structurer l'état est de le garder DRY (Don't Repeat Yourself). Déterminez la représentation minimale absolue de l'état dont votre application a besoin et calculez tout le reste à la demande. Par exemple, si vous construisez une liste de courses, vous pouvez stocker les articles sous forme de tableau dans l'état. Si vous souhaitez également afficher le nombre d'articles dans la liste, ne stockez pas le nombre d'articles comme une autre valeur d'état - à la place, lisez la longueur de votre tableau.

Réfléchir en React

Étape 4 : Déterminer ou mettre l'état « state »

Après avoir identifié les données d'état minimales de votre application, vous devez identifier le composant responsable de la modification de cet état, ou propriétaire de l'état. Rappelez-vous : React utilise un flux de données unidirectionnel, transmettant les données vers le bas de la hiérarchie des composants, du composant parent au composant enfant. Il se peut que l'on ne sache pas immédiatement quel composant doit posséder quel état. Cela peut s'avérer difficile si vous ne connaissez pas ce concept, mais vous pouvez le découvrir en suivant ces étapes, pour chaque élément d'état de votre application :

1. Identifiez chaque élément qui effectue un rendu en fonction de cet état.
2. Trouvez le composant parent commun le plus proche, c'est-à-dire un composant situé au-dessus de tous les autres dans la hiérarchie.
3. Décidez de l'emplacement de l'état :
 1. Souvent, vous pouvez placer l'état directement dans leur parent commun.
 2. Vous pouvez également placer l'état dans un composant situé au-dessus de leur parent commun.
 3. Si vous ne parvenez pas à trouver un composant dans lequel il est logique de placer l'état, créez un nouveau composant uniquement destiné à contenir l'état et ajoutez-le quelque part dans la hiérarchie, au-dessus du composant parent commun.

Étape 5 : Ajouter le flux de données inverse

L'états « states » circulent vers le bas de la hiérarchie. Mais pour modifier l'état en fonction des entrées de l'utilisateur, vous devrez prendre en charge le flux de données dans l'autre sens.